

ackee
blockchain security

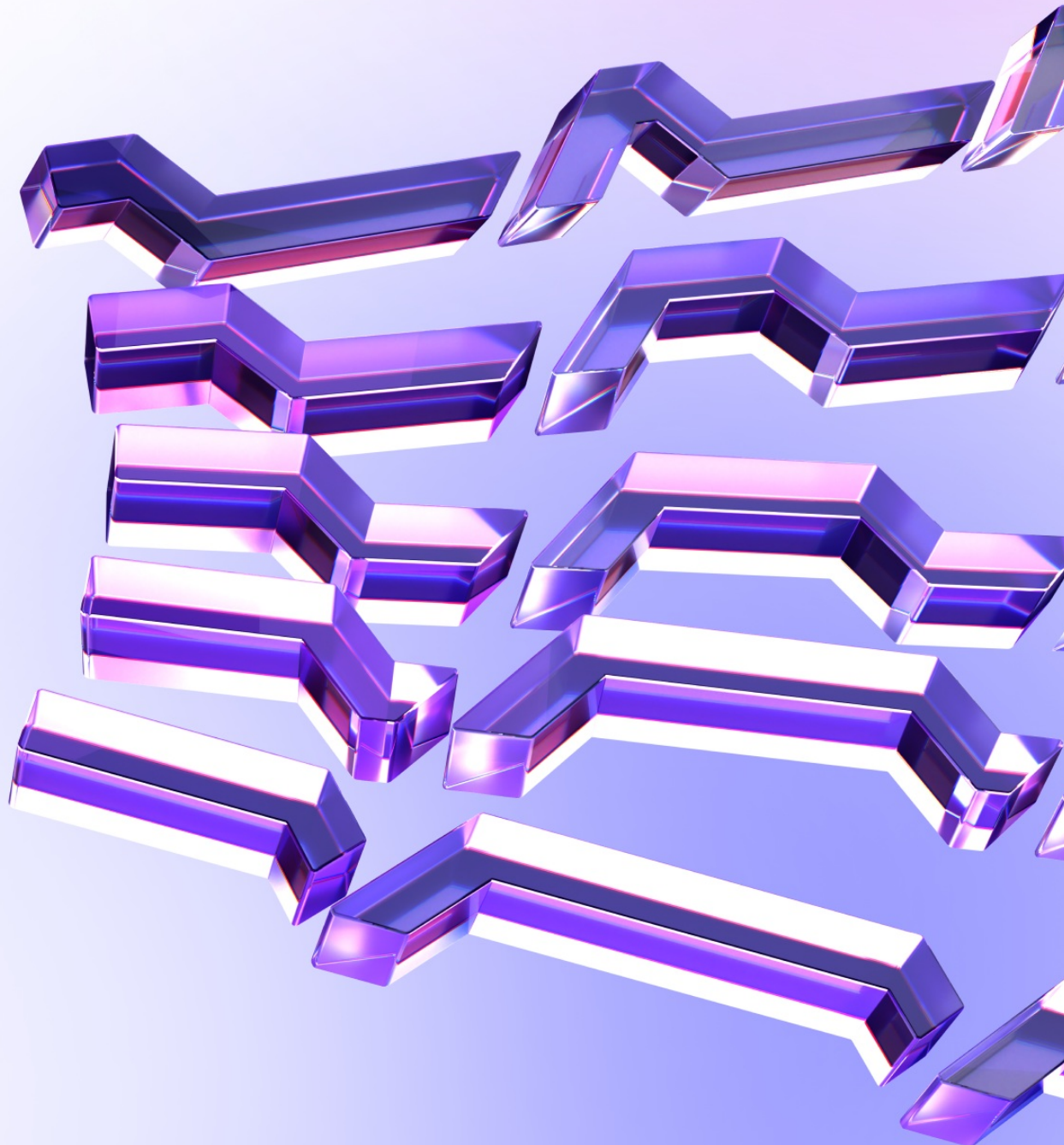


monerium

smart-contracts

December 11, 2025 19:56 UTC

Wake Arena AI Report



Contents

- 1. Document Revisions 3
- 2. Overview 4
 - 2.1. Ackee Blockchain Security 4
 - 2.2. Wake Arena 5
 - 2.3. Disclaimer 5
 - 2.4. Finding Classification 6
- 3. Executive Summary 7
 - Revision 1.0 7
- 4. Findings Summary 10
- Report Revision 1.0 12
 - Findings 12
- Appendix A: How to cite 32

1. Document Revisions

1.0	AI Scan	Dec 11, 2025 19:56 UTC
---------------------	---------	------------------------

2. Overview

This document presents the findings [Wake Arena](#) identified using AI vulnerability checks in the reviewed contracts.

2.1. Ackee Blockchain Security

Ackee Blockchain Security is an in-house team of security researchers performing security audits focusing on manual code reviews with extensive fuzz testing for Ethereum and Solana. Ackee is trusted by top-tier organizations in web3, securing protocols including Lido, Safe, and Axelar.

We develop open-source security and developer tooling [Wake](#) for Ethereum and [Trident](#) for Solana, supported by grants from Coinbase and the Solana Foundation. Wake and Trident help auditors in the manual review process to discover hardly recognizable edge-case vulnerabilities.

Our team teaches about blockchain security at the Czech Technical University in Prague, led by our co-founder and CEO, Josef Gattermayer, Ph.D. As the official educational partners of the Solana Foundation, we run the [School of Solana](#) and the [Solana Auditors Bootcamp](#).

Ackee's mission is to build a stronger blockchain community by sharing our knowledge.

Ackee Blockchain a.s.

Rohanske nabrezi 717/4

186 00 Prague, Czech Republic

<https://ackee.xyz>

hello@ackee.xyz

2.2. Wake Arena

This report was created by [Wake Arena](#), an automated vulnerability analysis tool. It uses [Wake](#) Framework with additional detectors for AI and static analysis of Solidity smart contracts.

To find potential vulnerabilities, Wake uses:

- The code structure and patterns
- Control flow graphs
- Data flow graphs
- Common vulnerability patterns
- Contract interactions

Wake Arena aims for precision, not recall. This means the presented findings are optimized for true positives, not for finding all possible issues, and should always be complemented with additional manual review for a complete security assessment.

2.3. Disclaimer

We've put our best effort to find known vulnerabilities in the system, but automated findings shouldn't be considered a complete list of all existing issues. The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

2.4. Finding Classification

Each finding is classified by two independent ratings: *Impact* and *Confidence*.

Impact

Measuring the potential consequences of the issue on the system.

- **Critical** - Code that activates the issue directly enables theft of significant assets with minimal preconditions.
- **High** - Code that activates the issue will lead to undefined or catastrophic consequences for the system.
- **Medium** - Code that activates the issue will result in consequences of serious substance.
- **Low** - Code that activates the issue will have outcomes on the system that are either recoverable or don't jeopardize its regular functioning.
- **Warning** - The issue represents a potential security concern in the code structure or logic that could become problematic with code modifications.
- **Info** - The issue relates to code quality practices that may affect security. Examples include insufficient logging for critical operations or inconsistent error handling patterns.

Confidence

Indicating the probability that the identified issue is a valid security concern.

- **High** - The analysis has identified a pattern that strongly indicates the presence of the issue.
- **Medium** - Evidence suggests the issue exists, but manual verification is recommended.
- **Low** - Potential indicators of the issue have been detected, but there is a significant possibility of false positives.

3. Executive Summary

Revision 1.0

Audit Overview

The audit of monerium/smart-contracts at commit

07788b768b461e7a6ccfd9d501f6ef69bc5f9db1 focused on the `SwapV1V2` upgradeable contract that mediates 1:1 swaps between distinct tokens `V1` and `V2`, with LitePSM-compatible entry points. The contract inherits OpenZeppelin upgradeable components (`UUPSUpgradeable`, `OwnableUpgradeable`, `ReentrancyGuardUpgradeable`) and uses `SafeERC20` for transfers. A total of 7 AI-detected issues were identified across the scoped file (`src/SwapV1V2.sol`).

Key observations

- 0 critical, 5 high, 0 medium, 0 low, 1 warning, 1 info; 7 total AI-detected issues (counted from issues/*.yaml).
- The most severe findings are five high-severity logic flaws where execution becomes a no-op when the caller equals the recipient, yet the function still emits `Swapped` and returns the input amount. This enables spoofed "successful" swaps or sales/purchases that can mislead downstream accounting, volume tracking, or incentive systems:
- `swapExactIn` at `src/SwapV1V2.sol:74-82` skips both `safeTransferFrom(tokenIn)` and `safeTransfer(tokenOut)` when `msg.sender == to`.
- `swapWithPermitStrict` at `src/SwapV1V2.sol:104-121` skips `permit`, `safeTransferFrom`, and `safeTransfer` when `msg.sender == to`.
- `swapWithPermitBestEffort` at `src/SwapV1V2.sol:140-172` skips optional `permit` and both transfers when `msg.sender == to`.
- `sellGem` at `src/SwapV1V2.sol:197-201` skips pull/push when `msg.sender ==`

`usr.`

- `buyGem` at `src/SwapV1V2.sol:217-221` skips pull/push when `msg.sender == usr.`
- Additional concerns include: a best-effort `permit` ABI encoding mismatch (expects 97-byte `abi.encodePacked` while the comment states `abi.encode`), and strict `permit` usage that hard-codes `owner = msg.sender`, limiting aggregator and relayer compatibility.

Overall, the codebase demonstrates generally sound use of established libraries and upgrade patterns, but the identified no-op pathways materially weaken security and correctness because events and return values can signal success without asset movement. Until these logic paths are removed or explicitly reverted and the permit interface clarified, the system should be considered not production-ready.

Key Technical Findings

- **High** - No-op swap path in `swapExactIn` enables spoofed success when `msg.sender == to`; both `safeTransferFrom(tokenIn)` and `safeTransfer(tokenOut)` are skipped while still emitting `Swapped` and returning `amountIn` (`src/SwapV1V2.sol:74-82`). Action: always perform transfers or revert when caller equals recipient; consider post-conditions on balance deltas.
- **High** - No-op path in `swapWithPermitStrict` when `msg.sender == to` bypasses `permit`, `safeTransferFrom`, and `safeTransfer`, yet still emits `Swapped` and returns `amountIn` (`src/SwapV1V2.sol:104-121`). Action: remove the conditional or revert; optionally assert balance changes after transfers.
- **High** - No-op path in `swapWithPermitBestEffort` when `msg.sender == to` bypasses the optional 2612 `permit` and both transfers while still signaling success via event and return value (`src/SwapV1V2.sol:140-172`). Action: remove the conditional or revert; keep best-effort `permit` but ensure real token movement.

- **High** - No-op PSM sell in `sellGem` when `msg.sender == usr` skips both `IERC20(V1).safeTransferFrom` and `IERC20(V2).safeTransfer` while emitting `Swapped` and returning `gemAmt (src/SwapV1V2.sol:197-201)`. Action: always execute transfers or revert on `msg.sender == usr`; verify balance deltas.
- **High** - No-op PSM buy in `buyGem` when `msg.sender == usr` skips both `IERC20(V2).safeTransferFrom` and `IERC20(V1).safeTransfer` while emitting `Swapped` and returning `gemAmt (src/SwapV1V2.sol:217-221)`. Action: always execute transfers or revert on `msg.sender == usr`; verify balance deltas.
- **Warning** - `swapWithPermitBestEffort` comment indicates `abi.encode(deadline, v, r, s)` but implementation accepts only a 97-byte `abi.encodePacked` layout and silently skips `permit` otherwise (`src/SwapV1V2.sol:134-148`). Action: align documentation and decoding, or support both 97- and 128-byte encodings; optionally revert on unexpected length.
- **Info** - `swapWithPermitStrict` hard-codes `owner = msg.sender` for ERC-2612 `permit`, breaking aggregator/relayer flows where the caller submits a user-signed permit (`src/SwapV1V2.sol:88-125`). Action: either document the constraint or add an explicit `owner` parameter and use it consistently in `permit` and `transferFrom`.

Conclusion

Despite leveraging robust OpenZeppelin upgradeable primitives and `SafeERC20`, the presence of five high-severity logic flaws that allow "successful" no-op executions means the current codebase is not ready for deployment. These issues can distort accounting and trigger downstream actions without asset movement, which is unacceptable for a swapping/PSM component. Remediating the caller-equals-recipient branches and clarifying `permit` handling should be prioritized; after fixes and re-audit, the security posture can be re-evaluated for production readiness.

4. Findings Summary

Summary of findings:

Critical	High	Medium	Low	Warning	Info	Total
0	5	0	0	1	1	7

Table 1. Findings Count by Impact

Findings in detail:

Finding title	Impact	Reported	Status
H1 : buyGem allows no-op when caller equals recipient (msg.sender == usr)	High	1.0	Reported
H2 : swapExactIn allows no-op when caller equals recipient (msg.sender == to)	High	1.0	Reported
H3 : sellGem allows no-op when caller equals recipient (msg.sender == usr)	High	1.0	Reported
H4 : swapWithPermitBestEffort allows no-op when caller equals recipient (msg.sender == to)	High	1.0	Reported
H5 : swapWithPermitStrict allows no-op when caller equals recipient (msg.sender == to)	High	1.0	Reported
W1 : swapWithPermitBestEffort comment mismatches ABI encoding; function only accepts 97-byte abi.encodePacked payload	Warning	1.0	Reported

Finding title	Impact	Reported	Status
M1 : swapWithPermitStrict hardcodes owner = msg.sender, preventing relayed permits and breaking aggregator flows	Info	1.0	Reported

Table 2. Table of Findings

Report Revision 1.0

Findings

The following section presents the list of findings discovered in this revision.
For the complete list of all findings, [Go back to Findings Summary](#)

H1: buyGem allows no-op when caller equals recipient (msg.sender == usr)

High impact finding

Impact:	High	Confidence:	High
Target:	./src/SwapV1V2.sol	Type:	Logic error

Description

The LitePSM-compatible `buyGem` function skips both `IERC20(V2).safeTransferFrom` and `IERC20(V1).safeTransfer` when the caller is also the recipient (`msg.sender == usr`).

Since `V1` and `V2` are different tokens, skipping both transfers yields no token exchange while still emitting `Swapped` and returning `gemAmt`. This violates the expected behavior of buying `V1` with `V2` and can mislead integrators that trust the event/return value.

Listing 1. Code snippet from src/SwapV1V2.sol

```
217 // if the caller is also the recipient, skip the transferFrom/transfer
218 if (msg.sender != usr) {
219     IERC20(V2).safeTransferFrom(msg.sender, address(this), gemAmt);
220     IERC20(V1).safeTransfer(usr, gemAmt);
221 }
222
223 emit Swapped(msg.sender, V2, V1, gemAmt, usr);
224 return gemAmt;
```

Exploit Scenario

A caller can create a buy that appears successful without moving tokens by setting themselves as the recipient.

Consider the following attack scenario:

1. Alice calls `buyGem` with `usr = Alice` and any non-zero `gemAmt` (no approval or balance is required because the branch is skipped).
2. The branch guarded by `if (msg.sender != usr)` is not taken, so neither `transferFrom` nor `transfer` executes.
3. The function still emits `Swapped` and returns `gemAmt`, which naive systems may treat as a successful purchase.
4. Bob's protocol records volume, distributes incentives, or triggers downstream actions from the event/return value, leading to financial or operational impact despite no token delivery.

Recommendation

Ensure `buyGem` performs real token transfers or rejects ambiguous calls.

Fix the vulnerability by either:

- Always executing `safeTransferFrom(V2)` and `safeTransfer(V1)` regardless of `msg.sender == usr`; or
- Reverting when `msg.sender == usr` to avoid spoofable success signals.

Optionally, enforce post-conditions by comparing balances before/after to assert that `usr` received `gemAmt` of `V1` and `msg.sender` provided `gemAmt` of `V2`.

[Go back to Findings Summary](#)

H2: swapExactIn allows no-op when caller equals recipient (msg.sender == to)

High impact finding

Impact:	High	Confidence:	High
Target:	./src/SwapV1V2.sol	Type:	Logic error

Description

The function `swapExactIn` conditionally skips both `IERC20(tokenIn).safeTransferFrom` and `IERC20(tokenOut).safeTransfer` when the caller is also the recipient (`msg.sender == to`).

Because `v1` and `v2` are different tokens (enforced by `initialize` and `_checkPair`), skipping both transfers yields no token movement while still emitting `Swapped` and returning `amountIn`. This violates expected swap semantics and can mislead integrators that trust the event or return value rather than checking balances.

State variables involved: `v1` and `v2` (both `address`). Initialization requires `v1 != v2`, and `_checkPair` ensures the pair is exactly `(v1, v2)` or `(v2, v1)`.

Listing 2. Code snippet from src/SwapV1V2.sol

```
74 // if the caller is also the recipient, skip the transferFrom/transfer
75 if (msg.sender != to) {
76     IERC20(tokenIn).safeTransferFrom(
77         msg.sender,
78         address(this),
79         amountIn
80     );
81     IERC20(tokenOut).safeTransfer(to, amountIn);
82 }
83
84 emit Swapped(msg.sender, tokenIn, tokenOut, amountIn, to);
85 return amountIn;
```

Exploit Scenario

A malicious caller can manufacture a swap that appears successful on-chain without moving any tokens by setting the recipient equal to themselves.

Consider the following attack scenario:

1. Alice crafts a call to `swapExactIn` with `to = Alice`, a valid `(tokenIn, tokenOut)` pair, and any non-zero `amountIn` (no approval or balance is required because `transferFrom` is skipped when `to == msg.sender`).
2. Alice submits the transaction; the `if (msg.sender != to)` branch is not entered, so no `transferFrom` or `transfer` executes.
3. The contract still emits `Swapped` and returns `amountIn`, which off-chain systems or naive integrators may interpret as a successful swap/fill.
4. Bob's protocol records a successful swap based on the event/return value and may issue credits, rebates, or trigger downstream actions, resulting in financial or accounting impact without any actual token delivery.

Recommendation

Remove the special-case that skips both transfers and enforce real token movement or explicit rejection when the caller is also the recipient.

Fix the vulnerability by either:

- Always executing both `safeTransferFrom(tokenIn)` and `safeTransfer(tokenOut)` regardless of `msg.sender == to` (recommended in this 1:1 venue since `tokenIn != tokenOut` by design); or
- Reverting when `msg.sender == to` to avoid ambiguous semantics and spoofable events/return values.

Additionally, consider post-conditions to harden integrations:

- Assert that `to` receives `amountIn` of `tokenOut` and that `amountIn` of `tokenIn`

was pulled from `msg.sender` (e.g., by comparing balances before/after), and

- Keep using `SafeERC20` and the checks-effects-interactions pattern to minimize integration risks.

[Go back to Findings Summary](#)

H3: sellGem allows no-op when caller equals recipient (msg.sender == usr)

High impact finding

Impact:	High	Confidence:	High
Target:	./src/SwapV1V2.sol	Type:	Logic error

Description

The LitePSM-compatible `sellGem` function skips both `IERC20(V1).safeTransferFrom` and `IERC20(V2).safeTransfer` if the caller is also the recipient (`msg.sender == usr`).

Given `V1` and `V2` are distinct tokens by design (`initialize` enforces `v1 != v2`), this branch produces no token movement while still emitting `Swapped` and returning `gemAmt`, violating the expected semantics of selling `V1` for `V2`.

Listing 3. Code snippet from `src/SwapV1V2.sol`

```
197 // if the caller is also the recipient, skip the transferFrom/transfer
198 if (msg.sender != usr) {
199     IERC20(V1).safeTransferFrom(msg.sender, address(this), gemAmt);
200     IERC20(V2).safeTransfer(usr, gemAmt);
201 }
202
203 emit Swapped(msg.sender, V1, V2, gemAmt, usr);
204 return gemAmt;
```

Exploit Scenario

A caller can fabricate a successful sale without transferring any tokens by setting the recipient to themselves.

Consider the following attack scenario:

1. Alice calls `sellGem` with `usr = Alice` and any non-zero `gemAmt` (no approval

or balance is required because the branch with `transferFrom` is skipped).

2. Because `msg.sender == usr`, the contract executes neither the pull of `V1` nor the push of `V2`.
3. The function still emits `Swapped` and returns `gemAmt`, which external systems may treat as a successful sale.
4. Bob's protocol updates accounting or distributes incentives based on the event/return value despite zero token delivery, causing financial or operational impact.

Recommendation

Always execute the token transfers in `sellGem` or reject calls where `msg.sender == usr`.

Fix the vulnerability by either:

- Unconditionally calling `safeTransferFrom(V1)` and `safeTransfer(V2)`; or
- Reverting when `msg.sender == usr` to prevent spoofable success signals.

For additional safety, consider asserting post-conditions:

- Verify balance deltas: `usr` increases by `gemAmt` of `V2` and `msg.sender` decreases by `gemAmt` of `V1`.

[Go back to Findings Summary](#)

H4: swapWithPermitBestEffort allows no-op when caller equals recipient (msg.sender == to)

High impact finding

Impact:	High	Confidence:	High
Target:	./src/SwapV1V2.sol	Type:	Logic error

Description

The function `swapWithPermitBestEffort` places its optional `permit` invocation and both token transfers inside a branch that is skipped if the caller equals the recipient (`msg.sender == to`).

Because the venue guarantees `tokenIn != tokenOut` (`initialize` enforces `v1 != v2` and `_checkPair` validates the pair), skipping both transfers yields a no-op while still emitting `Swapped` and returning `amountIn`. This breaks expected swap semantics and can mislead integrators that rely on the event/return value.

Listing 4. Code snippet from `src/SwapV1V2.sol`

```
140 // if the caller is also the recipient, skip the transferFrom/transfer
141 if (msg.sender != to) {
142     if (permitCalldata.length == 97) {
143         // Manually decode packed data: 32 bytes deadline + 1 byte v + 32
        bytes r + 32 bytes s
144         uint256 deadline = uint256(bytes32(permitCalldata[0:32]));
145         uint8 v = uint8(permitCalldata[32]);
146         bytes32 r = bytes32(permitCalldata[33:65]);
147         bytes32 s = bytes32(permitCalldata[65:97]);
148         // low-level call so non-2612 tokens don't revert the whole tx
149         // solhint-disable-next-line avoid-low-level-calls
150         (bool success, ) = tokenIn.call(
151             abi.encodeWithSelector(
152                 IERC20Permit.permit.selector,
153                 msg.sender,
154                 address(this),
155                 amountIn,
```

```

156         deadline,
157         v,
158         r,
159         s
160     )
161 };
162 // Intentionally ignore success - this is best effort
163 success; // Acknowledge the variable to avoid unused variable
    warning
164 }
165
166 IERC20(tokenIn).safeTransferFrom(
167     msg.sender,
168     address(this),
169     amountIn
170 );
171 IERC20(tokenOut).safeTransfer(to, amountIn);
172 }
173
174 emit Swapped(msg.sender, tokenIn, tokenOut, amountIn, to);
175 return amountIn;

```

Exploit Scenario

An attacker can generate a successful-looking swap without any token movement by setting the recipient to themselves.

Consider the following attack scenario:

1. Alice calls `swapWithPermitBestEffort` with `to = Alice`, a valid pair, and a non-zero `amountIn` (no approval or balance is required because the `transferFrom` is skipped).
2. The branch guarded by `if (msg.sender != to)` is not executed, so any best-effort `permit` and both transfers are bypassed.
3. The function still emits `Swapped` and returns `amountIn`, which naive consumers may interpret as a successful swap.
4. Bob's protocol credits volume or issues rewards based on the event/return value, resulting in financial or operational impact despite no assets

changing hands.

Recommendation

Do not skip both transfers when `msg.sender == to`. Enforce real token movement or reject the call to prevent spoofable success.

Fix the vulnerability by either:

- Always performing `safeTransferFrom(tokenIn)` and `safeTransfer(tokenOut)` regardless of `msg.sender == to`; or
- Reverting when `msg.sender == to`.

Optionally add post-conditions for stronger guarantees:

- Compare balances before and after to assert that `to` received `amountIn` of `tokenOut` and that `msg.sender` provided `amountIn` of `tokenIn`.

[Go back to Findings Summary](#)

H5: swapWithPermitStrict allows no-op when caller equals recipient (msg.sender == to)

High impact finding

Impact:	High	Confidence:	High
Target:	./src/SwapV1V2.sol	Type:	Logic error

Description

The function `swapWithPermitStrict` wraps both the ERC-2612 `permit`, the `safeTransferFrom(tokenIn)`, and the `safeTransfer(tokenOut)` inside a conditional that is skipped when the caller is also the recipient (`msg.sender == to`).

Since `v1` and `v2` are distinct tokens by construction (`initialize` requires `v1 != v2`, `_checkPair` enforces the pair), this branch results in no asset movement while still emitting `Swapped` and returning `amountIn`. Off-chain systems that trust the event or return value may treat such calls as successful swaps.

Listing 5. Code snippet from `src/SwapV1V2.sol`

```
104 // if the caller is also the recipient, skip the transferFrom/transfer
105 if (msg.sender != to) {
106     IERC20Permit(tokenIn).permit(
107         msg.sender,
108         address(this),
109         amountIn,
110         deadline,
111         v,
112         r,
113         s
114     );
115     IERC20(tokenIn).safeTransferFrom(
116         msg.sender,
117         address(this),
118         amountIn
119     );
120     IERC20(tokenOut).safeTransfer(to, amountIn);
```

```
121 }  
122  
123 emit Swapped(msg.sender, tokenIn, tokenOut, amountIn, to);  
124 return amountIn;
```

Exploit Scenario

A malicious caller can fabricate a swap that looks valid without moving tokens by choosing themselves as the recipient.

Consider the following attack scenario:

1. Alice prepares a call to `swapWithPermitStrict` with `to = Alice`, a valid pair, and non-zero `amountIn` (no approval, permit, or balance is needed on this path because the branch is skipped).
2. Alice submits the transaction; the `permit`, `transferFrom`, and `transfer` are all bypassed because `msg.sender == to`.
3. The contract still emits `Swapped` and returns `amountIn`, so external observers may accept this as a successful execution.
4. Bob's protocol updates accounting, accrues volume, or triggers payouts based on the event/return value, suffering financial or operational impact despite zero token delivery.

Recommendation

Eliminate the conditional that skips both transfers (and the `permit`) when `msg.sender == to`, or explicitly reject such calls.

Fix the vulnerability by either:

- Always attempting `permit` and executing `safeTransferFrom(tokenIn)` and `safeTransfer(tokenOut)` even when `msg.sender == to`; or
- Reverting when `msg.sender == to` to avoid spoofable success signals.

As an additional hardening step, assert post-conditions:

- Verify that `to` received `amountIn` of `tokenOut` and that `amountIn` of `tokenIn` was pulled from `msg.sender` by comparing balances; and
- Maintain the checks-effects-interactions pattern with `SafeERC20`.

[Go back to Findings Summary](#)

W1: swapWithPermitBestEffort comment mismatches ABI encoding; function only accepts 97-byte abi.encodePacked payload

Impact:	Warning	Confidence:	High
Target:	./src/SwapV1V2.sol	Type:	Code quality

Description

The function `swapWithPermitBestEffort` documents `permitCalldata` as `abi.encode(deadline, v, r, s)` or `empty`, but the implementation gates on an exact length of 97 bytes and manually slices the buffer as `32 bytes deadline + 1 byte v + 32 bytes r + 32 bytes s`. That layout corresponds to `abi.encodePacked(deadline, v, r, s)`, not `abi.encode(...)` (which would be 4 full words, i.e., 128 bytes, since `uint8 v` is padded to 32 bytes by the ABI encoder).

This discrepancy is a reliability footgun. If integrators follow the comment and pass `abi.encode(...)` (128 bytes), the 97-byte branch is never taken, the permit attempt is silently skipped, and the call proceeds to `safeTransferFrom`. Without a pre-existing allowance, `safeTransferFrom` reverts and the whole swap fails unexpectedly. The behavior thus depends on payload length rather than intent, which is surprising and difficult to diagnose in production.

Listing 6. Code snippet from `src/SwapV1V2.sol`

```
134         bytes calldata permitCalldata // abi.encode(deadline, v, r, s) or
        empty
135     ) external nonReentrant returns (uint256 amountOut) {
136         _checkPair(tokenIn, tokenOut);
137         if (amountIn == 0) revert ZeroAmount();
138         require(amountIn >= minOut, "slip");
139
140         // if the caller is also the recipient, skip the
        transferFrom/transfer
141         if (msg.sender != to) {
```

```

142         if (permitCalldata.length == 97) {
143             // Manually decode packed data: 32 bytes deadline + 1 byte v
+ 32 bytes r + 32 bytes s
144             uint256 deadline = uint256(bytes32(permitCalldata[0:32]));
145             uint8 v = uint8(permitCalldata[32]);
146             bytes32 r = bytes32(permitCalldata[33:65]);
147             bytes32 s = bytes32(permitCalldata[65:97]);
148             // low-level call so non-2612 tokens don't revert the whole
tx
149             // solhint-disable-next-line avoid-low-level-calls
150             (bool success, ) = tokenIn.call(
151                 abi.encodeWithSelector(
152                     IERC20Permit.permit.selector,
153                     msg.sender,
154                     address(this),
155                     amountIn,
156                     deadline,
157                     v,
158                     r,
159                     s
160                 )
161             );
162             // Intentionally ignore success - this is best effort
163             success; // Acknowledge the variable to avoid unused
variable warning
164         }

```

Affected function: `swapWithPermitBestEffort` (no persistent state variables are read or modified beyond event emission).

Exploit Scenario

A realistic failure mode arises when an integrator encodes the permit according to the comment rather than the implementation.

Consider the following attack scenario:

- Alice, an aggregator, builds `permitCalldata` as `abi.encode(deadline, v, r, s)` (128 bytes) using a signature from Bob for `tokenIn`;
- Alice calls `swapWithPermitBestEffort` expecting the contract to attempt the permit and then pull Bob's tokens;

- The contract checks `permitCalldata.length == 97`, which is false for `abi.encode(...)`, and silently skips the permit attempt; and
- The subsequent `safeTransferFrom` reverts because Bob has not pre-approved the contract, causing the route to fail and users to incur gas costs.

In production, this manifests as intermittent swap failures depending on whether integrators used `abi.encode` or `abi.encodePacked`, undermining the "best effort" property.

Recommendation

Make the encoding contract- and integrator-friendly by either aligning the documentation or supporting both encodings.

Fix the vulnerability by either:

- Updating the comment and integrator guidance to require `abi.encodePacked(deadline, v, r, s)`; or
- Supporting both formats: if `permitCalldata.length == 97`, decode packed as implemented; if `permitCalldata.length == 128`, decode with `abi.decode(permitCalldata, (uint256,uint8,bytes32,bytes32))` and then invoke `permit`.

Additionally, consider reverting on unexpected lengths to surface configuration errors sooner rather than silently skipping the permit attempt.

[Go back to Findings Summary](#)

M: swapWithPermitStrict hardcodes owner = msg.sender, preventing relayed permits and breaking aggregator flows

Impact:	Info	Confidence:	High
Target:	src/SwapV1V2.sol	Type:	Code quality

Description

The function `swapWithPermitStrict` constructs the ERC-2612 call with `owner` set to `msg.sender`. This design prevents standard relayer/aggregator patterns where a contract submits a user-signed permit on behalf of the user. When `owner != msg.sender`, the signature validation inside `permit` fails and the swap reverts.

This is not a direct loss-of-funds vulnerability, but it is a significant integration constraint that can cause production reverts and failed routes whenever a relayer or aggregator contract is the caller.

Listing 7. Code snippet from `src/SwapV1V2.sol`

```
106         IERC20Permit(tokenIn).permit(  
107             msg.sender,  
108             address(this),  
109             amountIn,  
110             deadline,  
111             v,  
112             r,  
113             s  
114         );
```

Affected function: `swapWithPermitStrict` (no persistent state variables are read or modified beyond event emission).

Exploit Scenario

A common integration breaks when a relayer contract submits a user-signed permit.

Consider the following attack scenario:

- Bob, a user, signs a 2612 permit authorizing the Swap contract to spend `amountIn` of his `tokenIn` until `deadline`;
- Alice, an aggregator contract, routes Bob's order and calls `swapWithPermitStrict` with Bob's `v`, `r`, `s` parameters;
- The function passes `owner = msg.sender` (Alice), not Bob, into `permit`, making the signature invalid; and
- The `permit` call reverts, the swap fails, and the aggregated route breaks, incurring gas costs and degrading UX.

In production, this blocks relayed flows unless the user calls the function directly or pre-approves, defeating the purpose of a strict-permit entry point.

Recommendation

Allow specifying the actual permit owner, or clearly document the constraint.

Fix the vulnerability by either:

- Adding an `address owner` parameter and using it in the `permit` call while still pulling funds from `owner` via `transferFrom` (signature correctness protects against misuse); or
- Keeping the current `owner = caller` design, but documenting prominently that only direct user calls are supported and aggregator/relayer usage will revert.

If adding `owner`, ensure the transfer path uses `owner` as the `from` in

`safeTransferFrom` for consistency with the permit.

[Go back to Findings Summary](#)

Appendix A: How to cite

Please cite this document as:

[Ackee Blockchain Security](#), Wake Arena Report | monerium: smart-contracts,
Wake Arena AI Report.



Thank You

Ackee Blockchain a.s.

Rohanske nabrezi 717/4
186 00 Prague
Czech Republic

hello@ackee.xyz

